Day 56 – API Management (APIM) – Part 5: Security Policies, Products & OAuth 2.0

Class date: 13th September 2026 | Module: SAP API Management – Verify API Key, Products/Applications, OAuth 2.0

One small naming clarification: item 7 in your policy list was written as **"OAuth v2.0 OSET"** (and you'd circled/crossed it as uncertain). The correct, official policy name is **"OAuth v2.0 SET"** (sets/updates custom attributes on an existing access token) – confirmed against SAP's own policy reference. Every other policy name in your list of 13 matched SAP's official security policy catalogue exactly.

1. Recap (start here first)

Basic Auth – **Encode, Decode** – used to get the **Username & Password** out of (or into) the **Authorization header.** For this, we use the **Basic Auth Policy.**

2. Security Policies – the full list

These are sometimes referred to as **generic policies** – reusable building blocks available on any API Proxy:

- **1.** Basic Authentication
- **2.** Decode JWT
- **3.** Generate JWT
- **4.** JSON Threat Protection
- **5.** OAuth v2.0
- **6.** OAuth v2.0 Get
- **7.** OAuth v2.0 SET
- **8.** Regular Expression Protection
- **9.** SAML Assertion Generation
- **10.** SAML Assertion Validation
- **11.** Verify API Key
- **12.** Verify JWT
- **13.** XML Threat Protection

Useful extra context: SAP's own recommended baseline template for most API Proxies combines **Verify API Key + JSON/XML Threat Protection + Regular Expression Protection + Spike Arrest + Quota** – a solid default starting point before adding anything more specific.

3. (11) Verify API Key

Verify API Key → the **apikey** can be supplied via the **URL, a Query Parameter, or a Header.** The policy's job is simply to **Verify** whether the supplied key is **correct or not.**

Step 1 – Product creation

Before Verify API Key can actually work, you need a **Product** to attach the key subscription to:

```
Configure -> APIs -> (existing flow) -> Edit -> Save -> Deploy
```

1. Go to **Engage** → **Create Product.**
2. Name the product, e.g. **"APIDemo."**
3. **Add** → select the **existing (API Proxy) Flow** to include in this Product.
4. **Publish** the Product. *(Practical tip from class: sometimes it takes clicking Publish more than once if it doesn't go through on the first try.)*
5. Confirm it shows **"Deployed Successfully."**

Only after the Product exists do you attach the **Verify API Key** policy onto the actual API Proxy.

4. Why Products exist – grouping similar APIs

A **Product** is simply a **place where you can group similar APIs** together.

Example: different APIs might belong to different business areas – e.g. **FICO APIs, SD APIs** (Finance/Controlling vs. Sales & Distribution, in SAP terms), or more generally **HR-related products** and **Finance-related products.** You **club all those relevant API Proxies into one location** – that location is the Product.

Navigation: **Engage** → **Product** → **grouping APIs.**

Developer Hub: this is where **all the APIs are managed** – the **Central Hub** is the **Developer Hub (all APIs), managed** centrally.

5. Subscribing to a Product – creating an Application

To actually **subscribe to** (consume) a Product, you need to **create an Application.**

Example: three different 3rd-party systems – **System A, System B, System C** – each want to consume APIs from a Product.

```
My Workspace -> Subscriptions -> (create an Application) -> Postman -> Application created -> Client ID & Password (API Key & API Secret) generated
```

Navigation (alternate path shown in class): **Engage** → **Subscriptions** → **Postman** → **API Key, API Secret.**

Example – two Products, two consuming systems

Product 1	Product 2
API 1, API 2, API 3, API 4, API 5	API 6, API 7, API 8, API 9, API 10
Consumed by: System A	Consumed by: System B

The API Key can be checked via a **query parameter**, or referenced directly as **request.header.apikey** when the key is sent in a header instead.

6. (5) OAuth v2.0

Scenario: Verify API Key relies on a **static** key. What if, instead, we want this **based upon OAuth 2.0** – how would we actually do that?

Important baseline fact: the **API Proxy URL by itself doesn't need credentials** to be called – just like the Day 54 demo showed (a proxy with no security policy attached simply passes requests through).

Security only applies once you attach a policy.

CPI uses OAuth 2.0 – how does APIM use OAuth 2.0?

You have to add a Policy. Specifically: **OAuth v2.0 policy** → **Verify Access Token.**

How it works: the Source sends its token in the **Authorization header**; APIM's **Verify Access Token** operation checks whether that token is **valid or not.**

Basic Auth vs. OAuth 2.0 – the core difference:

- **Basic Auth** → **Static** (username/password don't change on their own).
- **OAuth v2.0** → **Dynamic** (tokens **expire**, and then need to be **regenerated.**)

Generate Access Token

Navigation: **Create API Proxy** → **Policies** → **Generate Access Token** (an operation of the **OAuth v2.0** policy).

Grant Type options available: **Client Credentials, Authorization Code, Implicit Grant.** (These are the same standard OAuth 2.0 grant types introduced back on Day 41 for CPI – APIM's OAuth v2.0 policy supports configuring the proxy to issue tokens under any of these.)

Related operation also mentioned: **Validate Token** – confirming an already-issued token is still valid, separate from the initial Verify Access Token check on an incoming request.

Quick-recall summary

- 13 Security Policies in SAP API Management, including Basic Authentication, JWT (Generate/Verify/Decode), JSON/XML Threat Protection, OAuth v2.0 (+ Get, + SET), Regular Expression Protection, SAML Assertion (Generation/Validation), and Verify API Key.
- **Verify API Key** needs a **Product** created first (Engage → Create Product → add existing Flow → Publish → Deployed Successfully).
- **Products** group related API Proxies together (e.g. by business area: FICO, SD, HR, Finance) under the **Developer Hub**, the central place all APIs are managed.
- Consumers subscribe via an **Application** (My Workspace/Engage → Subscriptions), which generates an **API Key + API Secret** for that consumer.
- **OAuth v2.0** policy: **Verify Access Token** (checks an incoming token) vs. **Generate Access Token** (issues new tokens, with Grant Type = Client Credentials / Authorization Code / Implicit Grant).
- **Basic Auth = static** credentials; **OAuth 2.0 = dynamic** tokens that expire and must be regenerated.